Module 6 - Integrations with SIEM, EDR and Firewalls

6.1 - Integrating with SIEM Platforms using Python

1. Why Integrate with a SIEM?

A Security Information and Event Management (SIEM) system aggregates logs and events from across your environment, normalizes them, applies detection rules, and provides dashboards and alerts. Integrating your custom Python scripts with a SIEM lets you:

- **Automate queries** for indicators of compromise (IOCs) or anomalous patterns.
- **Push enriched events** (e.g. from your malware analysis pipelines) directly into your SIEM for correlation.
- **Build alerting pipelines** or dashboards programmatically.
- **Reduce manual effort** in threat hunting and incident response.

Popular SIEMs include **Splunk**, **Elastic SIEM (ELK)**, **IBM QRadar**, etc. Below we'll focus on Splunk (via its SDK and HTTP Event Collector), and briefly touch on Elastic.

2. Connecting to Splunk with the Python SDK

2.1 Installation

```
pip install splunk-sdk
```

2.2 Basic Concepts

- **Service**: Represents a connection to a Splunk instance (host, port, authentication).
- **Search Jobs**: Submit a search string (SPL), poll until it completes, then retrieve results.

2.3 Example: Authenticating and Running a Search

```python
import splunklib.client as client
import splunklib.results as results

# Create a Service instance to talk to Splunk
service = client.Service(
    host="splunk.example.com",
    port=8089,
    username="api_user",
    password="secret_password"
)

# Verify authentication
if not service.is_authenticated():
    raise Exception("Unable to authenticate to Splunk.")

# Create and run a search job
# This example searches for failed SSH logins in the last hour
search_query = 'search index=main sourcetype=linux_secure "Failed password" | stats count by src_ip'
job = service.jobs.create(search_query, exec_mode="blocking")

# Retrieve results
reader = results.JSONResultsReader(job.results())
for item in reader:
    if isinstance(item, dict):
        src_ip = item.get("src_ip")
        count  = item.get("count")
        print(f"Failed logins from {src_ip}: {count}")
```

Commentary

- We use `exec_mode="blocking"` for simplicity; for large searches prefer polling `job.is_done` in a loop.
- `JSONResultsReader` parses the raw JSON into Python dictionaries.

- Always handle authentication errors and timeouts in production code.

---

3. Querying Splunk via HTTP Event Collector (HEC)

Instead of using the SDK's port 8089, you can send raw events over HTTP to port 8088 with a token.

3.1 Setup in Splunk

1. Enable HEC in **Settings** → **Data Inputs** → **HTTP Event Collector**.
2. Create a new token with an index and source type.
3. Note your **HEC token** and endpoint URL (e.g. `https://splunk.example.com:8088/services/collector`).

3.2 Sending Events from Python

```
import requests
import json

HEC_URL = "https://splunk.example.com:8088/services/collector"
HEC_TOKEN = "YOUR_HEC_TOKEN"

def send_event(event_data, index="main", sourcetype="custom:python"):
    """
    Sends a single event to Splunk via HEC.
    event_data: dict or string
    """
    payload = {
        "index": index,
        "sourcetype": sourcetype,
        "event": event_data
    }
    headers = {
```

```
        "Authorization": f"Splunk {HEC_TOKEN}"
    }
    resp = requests.post(HEC_URL, headers=headers, json=payload, verify=True)
    if resp.status_code not in (200, 201, 202):
        raise Exception(f"HEC failed: {resp.status_code} {resp.text}")

# Example: send a structured alert
alert = {
    "alert_name": "Malware IOC Detected",
    "ioc_type": "ip",
    "ioc_value": "198.51.100.23",
    "severity": "high",
    "source": "python-script"
}
send_event(alert)
print("Event sent to Splunk.")
```

Commentary

- You can batch multiple events in one request by sending a list of payloads.
- Ensure your SSL certificates are valid (`verify=True`). In test environments you may disable with `verify=False` .

---

4. Connecting to Elastic SIEM (Elasticsearch) with Python

4.1 Installation

```
pip install elasticsearch
```

4.2 Basic Query Example

```python
from elasticsearch import Elasticsearch

# Connect to Elasticsearch
es = Elasticsearch(
    ["https://es.example.com:9200"],
    http_auth=("elastic", "changeme"),
    scheme="https",
    port=9200
)

# Query: find alerts with severity high in the last 24h
body = {
    "query": {
        "bool": {
            "must": [
                {"match": {"severity": "high"}},
                {"range": {"@timestamp": {"gte": "now-24h"}}}
            ]
        }
    }
}

resp = es.search(index="siem-alerts", body=body)
for hit in resp["hits"]["hits"]:
    src = hit["_source"].get("source_ip")
    msg = hit["_source"].get("message")
    print(f"{src}: {msg}")
```

Commentary

- Replace index name with your SIEM's alert index.
- You can use aggregations to build dashboards.

5. Best Practices

- **Authentication**: Prefer token-based auth (HEC, API keys) over basic auth.
- **Error Handling**: Catch network errors, handle HTTP timeouts, backoff on 429 responses.
- **Batching**: Send logs/events in bulk to reduce connection overhead.
- **Field Mapping**: Keep your field names consistent across sources for easier correlation.
- **Security**: Restrict your API tokens' permissions (read vs write).
- **Performance**: For large data volumes, use asynchronous requests (eg. `aiohttp`) or bulk APIs.

6.2 - Sending Events to SIEM (Syslog, HTTP, JSON, UDP)

1. Why Send Events to a SIEM?

A SIEM ingests security-relevant events from across your infrastructure. Your Python scripts can generate custom alerts or enrich existing logs—sending them to the SIEM allows:

- **Centralized correlation** of security events
- **Real-time alerting** on custom detections
- **Historical retention** and audit trails
- **Dashboarding** and reporting

Common transport formats include:

- **Syslog** (RFC 5424, UDP/TCP)
- **CEF** (Common Event Format)
- **LEEF** (Log Event Extended Format)
- **JSON over HTTP** (REST/HEC)

Below we cover each approach with Python examples and best practices.

2. Sending via Syslog

Syslog is a standard protocol for event forwarding. You can send to a local syslog daemon or direct to your SIEM's syslog receiver.

2.1 Python's `logging.handlers.SysLogHandler`

```
import logging
from logging.handlers import SysLogHandler

# Configure a logger that writes to a remote syslog collector
logger = logging.getLogger("myapp")
logger.setLevel(logging.INFO)

# UDP syslog (default RFC 3164)
syslog = SysLogHandler(
    address=("siem.example.com", 514),  # SIEM syslog listener
    facility=SysLogHandler.LOG_USER      # Choose appropriate facility
)
formatter = logging.Formatter(
    fmt="%(asctime)s %(hostname)s myapp[%(process)d]: %(message)s",
    datefmt="%b %d %H:%M:%S"
)
syslog.setFormatter(formatter)
logger.addHandler(syslog)

# Send a simple event
logger.info("ALERT: Malicious IP detected: 198.51.100.23")
```

Conceptual notes

- **Facility** categorizes source (e.g. LOG_AUTH, LOG_LOCAL0).

- **Severity** is derived from logging level (INFO→notice, WARNING→warn, ERROR→err).
- For **RFC 5424** support (structured data), use `SysLogHandler(socktype=socket.SOCK_STREAM)` and custom formatting.

---

3. Emitting CEF-Formatted Messages

CEF is a semi-structured format commonly used by appliances. Fields are pipe-delimited.

```
import socket
import time

def send_cef_event(host, port, cef_event):
    """
    Send a single CEF message over UDP.
    cef_event: full CEF string (including header)
    """
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.sendto(cef_event.encode(), (host, port))
    sock.close()

# Build a CEF header + extension
timestamp = time.strftime("%b %d %H:%M:%S")
cef_header = f"{timestamp} myhost CEF:0|MyCompany|MyProduct|1.0|100|Malicious IP Blocked|10|"
# Extension: key=value pairs separated by spaces
extension = "src=192.0.2.1 dst=198.51.100.23 suser=jdoe msg=\"Threat intelligence block\""

cef_message = cef_header + extension

# Send to SIEM syslog listener on UDP/514
send_cef_event("siem.example.com", 514, cef_message)
```

Conceptual notes

- CEF header fields: Vendor, Product, Version, EventID, Name, Severity.
- Extension keys must align with your SIEM's CEF field mapping.

---

4. Emitting LEEF-Formatted Messages

LEEF is IBM-style event format similar to CEF.

```
import socket

def send_leef_event(host, port, leef_event):
    """
    Send a single LEEF message over UDP or TCP.
    """
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.sendto(leef_event.encode(), (host, port))
    sock.close()

# Build a LEEF message
leef_header = "LEEF:2.0|MyCompany|MyProduct|1.0|100|"
leef_extension = "devTime=1620000000000 src=192.0.2.1 dst=198.51.100.23 usrName=jdoe msg=BlockedByPython"
leef_message = leef_header + leef_extension

send_leef_event("siem.example.com", 514, leef_message)
```

Conceptual notes

- LEEF uses key=value pairs but differs in header and delimiter conventions.
- Ensure your SIEM is configured to parse LEEF correctly.

5. Sending JSON Events over HTTP

Modern SIEMs often provide a REST ingestion endpoint (e.g., Splunk HEC, Elastic Ingest API).

5.1 Example: Splunk HTTP Event Collector (HEC)

```python
import requests

HEC_URL = "https://splunk.example.com:8088/services/collector/event"
HEC_TOKEN = "YOUR_TOKEN_HERE"

def send_json_event(event, index="main", sourcetype="_json"):
    """
    Send a JSON event to Splunk via HEC.
    event: Python dict or string
    """
    payload = {
        "time": time.time(),
        "host": "myhost",
        "index": index,
        "sourcetype": sourcetype,
        "event": event
    }
    headers = {"Authorization": f"Splunk {HEC_TOKEN}"}
    resp = requests.post(HEC_URL, json=payload, headers=headers, timeout=5)
    if resp.status_code not in (200, 201, 202):
        raise Exception(f"HEC error: {resp.status_code} {resp.text}")

# Send a structured JSON event
event_data = {
    "alert": "malicious_ip",
    "ip": "198.51.100.23",
```

```
    "severity": "high",
    "detected_by": "python_script"
}
send_json_event(event_data)
```

5.2 Example: Elastic Ingest API

```
from elasticsearch import Elasticsearch

es = Elasticsearch(["https://es.example.com:9200"], http_auth=("elastic","changeme"))

def send_elastic_event(index, document):
    """
    Index a single document in Elasticsearch.
    """
    resp = es.index(index=index, body=document)
    if resp.get("result") not in ("created", "updated"):
        raise Exception(f"Elasticsearch ingest error: {resp}")

# Send an event to the SIEM index
doc = {
    "@timestamp": time.strftime("%Y-%m-%dT%H:%M:%S"),
    "event_type": "malicious_ip",
    "ip": "198.51.100.23",
    "source": "python_script"
}
send_elastic_event("siem-alerts", doc)
```

Conceptual notes

- JSON over HTTP is firewall-friendly (port 80/443).
- It supports rich structured data and bulk ingestion.

- Handle retries, rate limits, and authentication securely.

---

6. Best Practices

1. **Format Consistency**: Use consistent field names, timestamps, and severity levels.
2. **Batching**: When possible, send events in batches to reduce connection overhead.
3. **Error Handling**: Implement retries, exponential backoff, and logging on failures.
4. **Security**: Protect API tokens—avoid hardcoding them; use environment variables or vaults.
5. **Testing**: Validate your messages with local syslog or sandbox SIEM before production.

6.3 - Interacting with EDRs via APIs (CrowdStrike, SentinelOne, XDRs)

1. Why Integrate Python with EDR APIs?

Endpoint Detection and Response (EDR) platforms—such as CrowdStrike Falcon, SentinelOne, and other XDRs—provide granular visibility into endpoint activity, automated detection, and remediation capabilities. By integrating Python scripts with these APIs, you can:

- **Automate incident response:** e.g., isolate compromised hosts, kill malicious processes.
- **Retrieve alerts and telemetry:** feed your own analysis pipelines or dashboards.
- **Enrich SIEM data:** correlate endpoint events with network or log data.
- **Orchestrate remediation:** create or update firewall rules, revoke certificates, deploy scripts.

---

2. EDR API Fundamentals

1. **Authentication**
   - CrowdStrike uses OAuth2 with client credentials (client ID/secret).
   - SentinelOne uses static API tokens.

2. **Rate Limiting**
   - Respect documented limits; implement back-off on HTTP 429.
3. **Versioning & Endpoints**
   - APIs evolve—pin your integration to a specific API version.
4. **Scopes & Permissions**
   - Grant only needed scopes (read alerts, write isolate, etc.).

---

3. Example: CrowdStrike Falcon API Integration

3.1 Obtain OAuth2 Token

```python
import requests

# Replace these with your CrowdStrike API credentials
CLIENT_ID     = "your_client_id"
CLIENT_SECRET = "your_client_secret"
OAUTH_URL     = "https://api.crowdstrike.com/oauth2/token"

def get_falcon_token(client_id, client_secret):
    """
    Obtain an OAuth2 token from CrowdStrike Falcon.
    """
    data = {
        'client_id': client_id,
        'client_secret': client_secret
    }
    headers = {'Content-Type': 'application/x-www-form-urlencoded'}
    response = requests.post(OAUTH_URL, data=data, headers=headers)
    response.raise_for_status()  # Raise on HTTP error
    token = response.json().get('access_token')
```

```
    return token

token = get_falcon_token(CLIENT_ID, CLIENT_SECRET)
print("Access token acquired.")
```

3.2 List Recent Detections

```
# Base URL for Falcon API endpoints
BASE_URL = "https://api.crowdstrike.com"

def list_detections(token, limit=10):
    """
    Query the Falcon detection summary endpoint.
    """
    url = f"{BASE_URL}/detects/queries/detects/v1"
    headers = {'Authorization': f"Bearer {token}"}
    params = {'limit': limit}
    # First, obtain list of detection IDs
    resp = requests.get(url, headers=headers, params=params)
    resp.raise_for_status()
    detect_ids = resp.json().get('resources', [])

    # Fetch details for those detection IDs
    details_url = f"{BASE_URL}/detects/entities/detects/v1"
    params = {'ids': detect_ids}
    resp = requests.get(details_url, headers=headers, params=params)
    resp.raise_for_status()
    return resp.json().get('resources', [])

# Example usage
detections = list_detections(token, limit=5)
```

```
for d in detections:
    print(f"Detection ID: {d['detect_id']}, Status: {d['status']}")
```

3.3 Isolate an Endpoint

```
def isolate_host(token, device_id):
    """
    Instruct Falcon to isolate a host by its device ID.
    """
    url = f"{BASE_URL}/devices/entities/devices-actions/v2"
    headers = {'Authorization': f"Bearer {token}", 'Content-Type': 'application/json'}
    body = {
        "device_ids": [device_id],
        "action_name": "isolate"
    }
    resp = requests.post(url, headers=headers, json=body)
    resp.raise_for_status()
    print(f"Isolation requested for device {device_id}")

# Example: isolate first affected host
if detections:
    example_device = detections[0].get('device_id')
    isolate_host(token, example_device)
```

---

4. Example: SentinelOne Management API Integration

4.1 Authentication (API Token)

```
import requests
```

```python
# Your SentinelOne API token and site URL
API_TOKEN = "your_sentinelone_token"
SITE_URL  = "https://usea1-partners.sentinelone.net/web/api/v2.1"

def sentinel_request(endpoint, params=None, method='GET', body=None):
    """
    Helper function to call SentinelOne API endpoints.
    """
    url = f"{SITE_URL}/{endpoint}"
    headers = {
        'Authorization': f"APIToken {API_TOKEN}",
        'Content-Type': 'application/json'
    }
    if method == 'GET':
        resp = requests.get(url, headers=headers, params=params)
    elif method == 'POST':
        resp = requests.post(url, headers=headers, json=body)
    else:
        raise ValueError("Unsupported HTTP method")

    resp.raise_for_status()
    return resp.json()

# Test connectivity
resp = sentinel_request("agents")
print(f"Total agents: {resp.get('data', {}).get('totalCount')}")
```

4.2 Retrieve Threats and Alerts

```python
def list_threats():
    """
    Retrieve recent threat records from SentinelOne.
    """
```

```
    resp = sentinel_request("threats", params={'limit': 10})
    return resp.get('data', [])

threats = list_threats()
for t in threats:
    print(f"Threat ID: {t['id']}, Status: {t['status']}, Type: {t['threatType']}")
```

4.3 Quarantine an Agent

```
def quarantine_agent(agent_id):
    """
    Quarantine (isolate) a SentinelOne agent by its ID.
    """
    endpoint = f"agents/{agent_id}/quarantine"
    resp = sentinel_request(endpoint, method='POST')
    print(f"Quarantine action: {resp.get('data')}")

# Example: quarantine the first agent in list
if threats:
    example_agent = threats[0]['agentId']
    quarantine_agent(example_agent)
```

---

5. Best Practices and Course-Level Insights

- **Token Security**: Store API keys in environment variables or secure vaults, never in code.
- **Error Handling**: Wrap API calls in try/except, handle 4xx/5xx HTTP status codes, implement retries with exponential back-off.
- **Rate Limits**: Respect provider limits—monitor `X-RateLimit-*` headers if provided.
- **Idempotency**: Design your remediation calls (isolate/quarantine) to be safe if inadvertently retried.
- **Data Modeling**: Normalize fields (timestamps, hostnames, user IDs) into a common schema for your pipelines.

- **Audit Trails**: Log each API action with full request/response for post-mortem and compliance.

6.4 - Automating IOC Blocking in Firewalls and EDR

1. Why Automate IOC Blocking?

Once you detect an Indicator of Compromise (IOC)—an IP, domain, URL or file hash—immediate blocking can prevent further compromise. Automating this step:

- **Reduces dwell time** by cutting off malicious infrastructure quickly.
- **Scales** to hundreds of IOCs without manual firewall edits.
- **Ensures consistency** across devices and endpoints.
- **Integrates** with threat feeds and detection pipelines for end-to-end automation.

Two common enforcement points are:

1. **Perimeter firewalls** (e.g. FortiGate, Palo Alto)
2. **Endpoint EDR platforms** (e.g. CrowdStrike, SentinelOne)

---

2. Firewall Integration Example: FortiGate via REST API

FortiGate's REST API allows you to programmatically update address objects and address groups used in security policies.

2.1 Prerequisites

- REST API enabled on the FortiGate
- An API user with permissions to modify firewall objects and policies
- Python package `fortiosapi` (or use raw `requests`)

```
pip install fortiosapi
```

2.2 Code: Add IP to Blocklist

```
from fortiosapi import FortiOSAPI

# Firewall connection details
FGT_HOST     = "https://firewall.example.com"
FGT_USERNAME = "api_user"
FGT_PASSWORD = "secure_password"

# The IOC to block
malicious_ip = "203.0.113.45"

# Name for the address object
object_name = f"IOC_{malicious_ip}"

# Name of the address-group used by your block policy
group_name = "IOC-Blacklist"

# 1. Authenticate
fos = FortiOSAPI()
fos.debug('off')  # turn on for troubleshooting
fos.login(FGT_HOST, FGT_USERNAME, FGT_PASSWORD)

# 2. Create or update a firewall address object for the IOC
fos.set(
    'firewall', 'address',
    mkey=object_name,
    data={
        'name': object_name,
        'subnet': f"{malicious_ip}/32",
```

```
        'type': 'ipmask',
        'visibility': 'enable'
    }
)

# 3. Ensure that address-object is a member of the block group
fos.set(
    'firewall', 'addrgrp',
    mkey=group_name,
    data={
        'name': group_name,
        'member': [{'name': object_name}]
    }
)

# 4. (Optional) Commit a policy change or trigger a reload if needed
#    Many FortiGate setups apply changes immediately.

# 5. Logout
fos.logout()

print(f"Blocked {malicious_ip} on FortiGate in group {group_name}")
```

Commentary

- `mkey` is the “managed key” (unique ID) for the object.
- We use `/32` subnet for a single IP.
- The `addrgrp` call appends or updates the address-group.
- Ensure a security policy references `IOC-Blacklist` in its destination address.

---

3. EDR Integration Example: CrowdStrike Network Containment

CrowdStrike Falcon supports **network containment**, isolating the endpoint's network traffic (except to the Falcon cloud). While not blocking individual IPs, it effectively prevents any further external communication.

3.1 Obtain OAuth2 Token

```python
import requests

CLIENT_ID     = "your_client_id"
CLIENT_SECRET = "your_client_secret"
OAUTH_URL     = "https://api.crowdstrike.com/oauth2/token"

def get_falcon_token():
    data = {'client_id': CLIENT_ID, 'client_secret': CLIENT_SECRET}
    headers = {'Content-Type': 'application/x-www-form-urlencoded'}
    resp = requests.post(OAUTH_URL, data=data, headers=headers)
    resp.raise_for_status()
    return resp.json()['access_token']

token = get_falcon_token()
```

3.2 Network Containment (Isolation)

```python
def contain_host(device_id, expire_seconds=3600):
    """
    Place an endpoint into network containment for a duration.
    """
    url = "https://api.crowdstrike.com/devices/entities/containment/v1"
    headers = {
        'Authorization': f"Bearer {token}",
        'Content-Type': 'application/json'
```

```
    }
    body = {
        "device_ids": [device_id],
        "parameters": {"expire_in": expire_seconds}
    }
    resp = requests.post(url, headers=headers, json=body)
    resp.raise_for_status()
    print(f"Host {device_id} contained for {expire_seconds} seconds")

# Example usage
example_device_id = "abcdef12-3456-7890-abcd-ef1234567890"
contain_host(example_device_id)
```

Commentary

- CrowdStrike's "containment" equals isolation at the network level.
- The `expire_in` parameter defines how long the isolation lasts.
- For true IOC blocking (specific IP/domain), you'd combine containment with firewall updates or EDR policy pushes.

---

4. Putting It Together: Automated IOC Pipeline

Below is a simplified pipeline that:

1. **Fetches** new malicious IPs from a threat feed (pseudo-code)
2. **Blocks** them on the firewall
3. **Isolates** hosts that communicated with them

```
import requests

THREAT_FEED_URL = "https://threat-feed.example.com/iocs.csv"
```

```python
def fetch_iocs():
    resp = requests.get(THREAT_FEED_URL)
    resp.raise_for_status()
    # Parse CSV of IPs
    return [line.strip() for line in resp.text.splitlines() if line]

def main():
    iocs = fetch_iocs()
    firewall = FortiOSAPI(); firewall.login(FGT_HOST, FGT_USERNAME, FGT_PASSWORD)
    cs_token = get_falcon_token()

    for ip in iocs:
        # 1. Block on Firewall
        addr_name = f"IOC_{ip}"
        firewall.set('firewall','address', mkey=addr_name, data={'name':addr_name,'subnet':f"
{ip}/32","type":"ipmask"})
        firewall.set('firewall','addrgrp', mkey=group_name, data={'name':group_name,'member':
[{'name':addr_name}]})
        print(f"Blocked {ip} at perimeter")

        # 2. Identify endpoints that have communicated with this IP
        #    (In practice query EDR telemetry; here we use a stub)
        affected_devices = lookup_devices_communicated_with(ip)

        # 3. Isolate those endpoints
        for dev in affected_devices:
            contain_host(dev, expire_seconds=7200)

    firewall.logout()

if __name__ == "__main__":
    main()
```

> `lookup_devices_communicated_with` would be implemented by querying EDR telemetry or SIEM logs for endpoints having outbound sessions to the IOC.

---

5. Best Practices

1. **Atomicity**: Ensure each block/unblock action can be reversed.
2. **TTL Management**: Automatically remove stale IOCs after a defined period.
3. **Logging/Audit**: Record all automated changes with timestamps and operators.
4. **Testing**: Validate IOC entries on a development device before wide rollout.
5. **Rate Limits**: When blocking hundreds of IOCs, respect API call limits.

6.5 - Building Custom Detection Pipelines

1. Overview

A detection pipeline is a sequence of stages that ingest raw data (logs, alerts, telemetry), normalize and enrich it, apply detection logic, and emit actionable outputs (alerts, SIEM events, tickets). Architecting a pipeline in Python teaches modular design, scalability, and maintainability.

Key stages:

1. **Ingestion** – pull or receive data.
2. **Normalization** – parse and convert to common schema.
3. **Enrichment** – add context (threat intelligence, asset tags).
4. **Detection** – apply rules, thresholds, ML models.
5. **Output** – generate alerts or forward to SIEM/EDR.

---

2. Pipeline Architecture (Conceptual)

```
[ Raw Data Sources ]
        ↓
   Ingestion Layer
        ↓
[ Normalizer / Parser ]
        ↓
   Enrichment Layer
        ↓
   Detection Engine
        ↓
    Alerting / Output
```

- **Decoupling**: Each stage is a function or class with clear input/output.
- **Batching**: Process records in batches for efficiency.
- **Error Handling**: Failures in enrichment or detection should not drop entire batch.
- **Configuration**: Rules, thresholds, API keys live in external YAML/JSON.

---

3. Example Pipeline in Python

```python
import json
import yaml
import requests
from datetime import datetime

# 3.1 Load configuration (rules, thresholds, API keys)
with open("pipeline_config.yaml") as cfg:
    config = yaml.safe_load(cfg)
```

```python
TI_API_KEY = config["threat_intel"]["api_key"]
SIEM_HEC = config["siem"]["hec_url"]
SIEM_TOKEN = config["siem"]["hec_token"]

# 3.2 Ingestion: read from a log file (could be Kafka, HTTP, etc.)
def ingest_logs(filepath):
    """
    Yield one log record (JSON) per line.
    """
    with open(filepath) as f:
        for line in f:
            try:
                yield json.loads(line)
            except json.JSONDecodeError:
                continue  # skip malformed

# 3.3 Normalization: map vendor fields to common schema
def normalize(record):
    """
    Convert raw record into standardized dict.
    Expected raw fields: timestamp, src_ip, dst_ip, event_type
    """
    return {
        "timestamp": datetime.fromisoformat(record["timestamp"]),
        "src_ip": record.get("src_ip"),
        "dst_ip": record.get("dst_ip"),
        "user": record.get("user", "").lower(),
        "event": record.get("event_type")
    }

# 3.4 Enrichment: add IP reputation
def enrich_ip(ip):
    """
```

```python
    Query an external Threat Intel API for reputation score.
    """
    url = f"https://api.abuseipdb.com/api/v2/check"
    headers = {"Accept": "application/json", "Key": TI_API_KEY}
    params = {"ipAddress": ip, "maxAgeInDays": 30}
    resp = requests.get(url, headers=headers, params=params, timeout=5)
    if resp.status_code == 200:
        data = resp.json().get("data", {})
        return data.get("abuseConfidenceScore", 0)
    return 0

# 3.5 Detection: simple threshold-based rule
def detect(record):
    """
    Return True if event should trigger alert.
    E.g., if IP abuse score exceeds threshold or repeated failures.
    """
    alerts = []
    score = enrich_ip(record["src_ip"])
    if score >= config["detection"]["ip_reputation_threshold"]:
        alerts.append(f"High abuse score: {record['src_ip']} = {score}")
    # Example: brute-force detection
    if record["event"] == "failed_login":
        # In real code, track counts per IP (e.g., in Redis)
        # Here we simulate with a single-event rule:
        alerts.append(f"Failed login: {record['src_ip']}")
    return alerts

# 3.6 Output: send alerts to SIEM via HEC
def send_to_siem(alerts, record):
    """
    Send a list of alert messages to SIEM HEC.
    """
    for msg in alerts:
```

```python
        payload = {
            "time": record["timestamp"].timestamp(),
            "host": "pipeline_host",
            "sourcetype": "custom:alerts",
            "event": {
                "message": msg,
                "src_ip": record["src_ip"],
                "event": record["event"]
            }
        }
        headers = {"Authorization": f"Splunk {SIEM_TOKEN}"}
        resp = requests.post(SIEM_HEC, json=payload, headers=headers, timeout=3)
        if resp.status_code not in (200, 201, 202):
            # handle failure (log, retry, dead-letter)
            print(f"HEC error: {resp.status_code} {resp.text}")

# 3.7 Orchestrator: tie stages together
def run_pipeline(logfile):
    """
    Main function to run the detection pipeline.
    """
    for raw in ingest_logs(logfile):
        norm = normalize(raw)
        alerts = detect(norm)
        if alerts:
            send_to_siem(alerts, norm)

if __name__ == "__main__":
    run_pipeline("raw_events.jsonl")
```

---

4. Deep-Dive Concepts

- **Stateful Detection**: For brute-force, maintain counters in external store (Redis, DB).
- **Batching & Parallelism**: Use Python's `concurrent.futures` or message queues for high throughput.
- **Circuit Breakers**: If TI API is down, fail open or use caching to avoid pipeline stall.
- **Schema Evolution**: Version your normalized schema to support new event types.
- **Testing**: Write unit tests for each stage (normalize/detect/enrich).

---

5. Scaling and Production

- **Containerization**: Dockerize the pipeline; use Kubernetes CronJobs or Airflow DAGs.
- **Observability**: Emit metrics (counts, latencies) to Prometheus or ELK.
- **Security**: Secure API credentials, use least privilege service accounts.
- **Resilience**: Implement retries, back-off strategies, and dead-letter queues for poison messages.

6.6 - Real-Time Alerting and Response Automation

1. Course-Level Concepts

- **Event-Driven vs. Polling**
  - *Polling* (e.g. looping over files with `time.sleep()`) introduces latency and wasted resources.
  - *Event-driven* uses OS hooks (file watchers, sockets) to react instantly when new data arrives.
- **Detection and Response Separation**
  - *Detection* identifies suspicious activity via patterns, thresholds, or ML models.
  - *Response* executes automated actions (notifications, containment, ticket creation) based on detection.
- **Idempotency and Rate-Limiting**
  - Ensure that repeated detections don't trigger duplicate responses.
  - Implement de-duplication windows (e.g. "don't alert on the same IP more than once every 5 minutes").
- **Integration Points**

    - **Notification**: Email, Slack/Webhook, SMS
    - **Orchestration**: SOAR platforms via REST or message bus
    - **Enforcement**: Firewall rule updates, EDR isolation calls
- **Reliability**
    - Use persistent queues (e.g. RabbitMQ, Kafka) for buffering events.
    - Graceful error handling and retry logic for downstream failures.

---

2. Example: File-Watch Tail + Alert + Response

Below is a Python example that:

1. **Watches** a log file in real time using `watchdog`.
2. **Applies** a regex to each new line for a critical pattern (e.g., repeated login failures).
3. **Rate-limits** alerts to once per IP per 5 minutes.
4. **Sends** a Slack notification via webhook.
5. **Triggers** a placeholder response function (e.g., block IP in firewall).

```python
import re
import time
import threading
from collections import defaultdict, deque
from datetime import datetime, timedelta

import requests
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

# Configuration
LOG_PATH       = "/var/log/auth.log"
```

```python
PATTERN        = re.compile(r"Failed password for .* from (?P<ip>\d+\.\d+\.\d+\.\d+)")
ALERT_WINDOW   = timedelta(minutes=5)  # suppress duplicate alerts per IP
SLACK_WEBHOOK  = "https://hooks.slack.com/services/XXXXX/XXXXX/XXXXX"

# In-memory store of last alert times per IP
last_alert_time = defaultdict(lambda: datetime.min)

def send_slack_notification(ip, count):
    """
    Send an alert to Slack via Incoming Webhook.
    """
    payload = {
        "text": f"Security Alert: {count} failed logins detected from {ip}"
    }
    resp = requests.post(SLACK_WEBHOOK, json=payload, timeout=3)
    resp.raise_for_status()

def respond_to_incident(ip):
    """
    Placeholder for automated response logic, e.g., adding firewall rule.
    """
    # Example: call firewall API to block IP
    # block_ip_on_firewall(ip)
    print(f"[{datetime.now()}] Responding by blocking IP {ip}")

class LogHandler(FileSystemEventHandler):
    """
    Watches for file modifications and processes new lines.
    """
    def __init__(self, path):
        super().__init__()
        self.path   = path
        self._inode = None
        self._file  = None
```

```python
        self._seek_to_end()

    def _seek_to_end(self):
        """
        Open file, remember inode, and seek to end so we read only new lines.
        """
        self._file = open(self.path, "r")
        self._inode = self._file.fileno()
        self._file.seek(0, 2)

    def on_modified(self, event):
        """
        Called whenever the watched file is written to.
        """
        if event.src_path != self.path:
            return
        # Read all new lines
        for line in self._file:
            self.process_line(line.strip())

    def process_line(self, line):
        """
        Apply detection logic to each line.
        """
        match = PATTERN.search(line)
        if not match:
            return
        ip = match.group("ip")
        now = datetime.now()

        # Suppress duplicate alerts within ALERT_WINDOW
        if now - last_alert_time[ip] < ALERT_WINDOW:
            return
```

```python
        # Count occurrences in the window (could be replaced with more complex state)
        last_alert_time[ip] = now
        # In real code, you might track counts in a deque for sliding-window logic

        # Trigger alert and response in separate threads to avoid blocking
        threading.Thread(target=send_slack_notification, args=(ip, 1), daemon=True).start()
        threading.Thread(target=respond_to_incident, args=(ip,), daemon=True).start()

def start_monitoring():
    """
    Initialize watchdog observer to monitor the log file.
    """
    event_handler = LogHandler(LOG_PATH)
    observer = Observer()
    observer.schedule(event_handler, path=LOG_PATH, recursive=False)
    observer.start()
    print(f"Started monitoring {LOG_PATH} for failed login patterns.")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
    observer.join()

if __name__ == "__main__":
    start_monitoring()
```

---

3. Key Takeaways

- **FileSystemEventHandler** from `watchdog` lets you react to file writes immediately.
- **Regex-based detection** is efficient for line-by-line log analysis.

- **Rate-limiting** via timestamps prevents alert storms.
- **Threading** offloads network calls (Slack, API) so the file watcher never blocks.
- **Separation of concerns**: detection, notification, and response logic live in separate functions for testability and maintainability.

6.7 - Custom Dashboards from SIEM/EDR Data

1. Why Build Custom Dashboards?

Out-of-the-box dashboards may not surface the exact metrics your team needs. Custom dashboards let you:

- **Tailor metrics** to your use cases (e.g. failed logins, top alert types)
- **Combine data sources** (SIEM + EDR + vulnerability scanner) in one view
- **Control refresh rates** and caching for performance
- **Embed interactive controls** (dropdowns, date pickers) for ad-hoc investigations

Key components of a dashboard application:

1. **Data Retrieval Layer**
   - Connects to SIEM/EDR APIs or databases
   - Applies aggregation queries
2. **Transformation Layer**
   - Converts raw query results into structures suitable for visualization
   - Handles paging, caching, error-handling
3. **Presentation Layer**
   - Renders interactive plots/tables in a web app
   - Provides filters and drill-down capabilities

---

2. Technology Choices

- **Backend**: Python libraries to query data (e.g. `elasticsearch`, `splunk-sdk`, `requests`)
- **Frontend Frameworks**:
  - **Plotly Dash** (Python-native)
  - **Streamlit** (rapid prototypes)
  - **Flask + React/Vue** (for full-stack control)

Below is an example using **Dash** and **Elasticsearch**:

---

3. Example: Dash App Visualizing Failed Login Counts from Elasticsearch

```python
# app.py
from datetime import datetime
from elasticsearch import Elasticsearch
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import pandas as pd

# 3.1 Data Retrieval: connect to Elasticsearch
es = Elasticsearch(
    ["https://es.example.com:9200"],
    http_auth=("elastic", "changeme"),
    scheme="https",
    port=9200
)

def query_failed_logins(start_time, end_time):
    """
    Query Elasticsearch for count of failed logins per source IP
    between start_time and end_time.
    Returns a pandas DataFrame.
```

```
    """
    body = {
        "size": 0,
        "query": {
            "bool": {
                "must": [
                    {"match": {"event_type": "failed_login"}},
                    {"range": {"@timestamp": {"gte": start_time, "lte": end_time}}}
                ]
            }
        },
        "aggs": {
            "by_src_ip": {
                "terms": {"field": "src_ip", "size": 10}
            }
        }
    }
    resp = es.search(index="security-logs", body=body)
    buckets = resp["aggregations"]["by_src_ip"]["buckets"]
    # Build DataFrame for Dash
    return pd.DataFrame({
        "src_ip": [b["key"] for b in buckets],
        "count":  [b["doc_count"] for b in buckets]
    })

# 3.2 Dash App Setup
app = Dash(__name__)
app.layout = html.Div([
    html.H1("Failed Logins Dashboard"),
    html.Div([
        html.Label("Start Time (ISO)"),
        dcc.Input(id="start-time", type="text",
                  value=(datetime.utcnow().replace(microsecond=0).isoformat() + "Z")),
        html.Label("End Time (ISO)"),
```

```
        dcc.Input(id="end-time", type="text",
                  value=(datetime.utcnow().replace(microsecond=0).isoformat() + "Z")),
        html.Button("Update", id="update-button")
    ]),
    dcc.Graph(id="failed-logins-graph")
])

# 3.3 Callback: update graph on button click
@app.callback(
    Output("failed-logins-graph", "figure"),
    Input("update-button", "n_clicks"),
    Input("start-time", "value"),
    Input("end-time", "value")
)
def update_graph(n_clicks, start_iso, end_iso):
    # Convert inputs to strings acceptable by Elasticsearch
    df = query_failed_logins(start_iso, end_iso)
    # Build a bar chart with Plotly Express
    fig = px.bar(
        df,
        x="src_ip", y="count",
        title=f"Top 10 Failed Logins from {start_iso} to {end_iso}",
        labels={"src_ip": "Source IP", "count": "Failed Login Count"}
    )
    return fig

# 3.4 Run the app
if __name__ == "__main__":
    app.run_server(host="0.0.0.0", port=8050, debug=True)
```

---

4. Fundamental Concepts Demonstrated

- **Separation of Concerns**
  - `query_failed_logins` handles data access and transformation.
  - Dash layout and callbacks manage presentation logic.
- **Interactive Controls**
  - Inputs for start/end times let analysts narrow time windows.
- **Efficient Aggregation**
  - Elasticsearch `terms` aggregation returns top N buckets without full scroll.
- **Performance Considerations**
  - In production, add caching (e.g. memoization) to avoid repeated API calls on every refresh.
  - Use asynchronous callbacks or background jobs for heavy queries.
- **Scalability**
  - Can extend with more graphs (heatmaps, time series).
  - Modularize further by splitting queries and components into separate Python modules.

---

5. Next Steps

- Add **authentication** to restrict dashboard access.
- Implement **auto-refresh** intervals for near real-time updates.
- Embed **additional data** (EDR alerts, vulnerability scores) in the same UI.
- Explore **deployment** via Docker and a production WSGI server (e.g. Gunicorn).

6.8 - Threat Hunting Automation with SIEM + Python

1. Conceptual Foundations

- **Threat Hunting vs. Alerting**
  - *Alerting* relies on predefined rules that fire on known indicators; often reactive.

    - *Threat hunting* is proactive, hypothesis-driven exploration of raw data to uncover unknown or stealthy threats before they generate alerts.
- **Hunting Hypotheses**
    - Formulate questions such as “Are there hosts exhibiting DNS tunneling?” or “Which users have executed PowerShell recently with unusual parameters?”
    - Each hypothesis maps to one or more search queries over log data.
- **Data Sources**
    - Centralized logs in SIEM (authentication, process execution, network flows).
    - Endpoint telemetry from EDR.
    - Threat intelligence feeds (IPs, domains, hashes).
- **Automation Benefits**
    - Runs hunts on schedule or on-demand without manual effort.
    - Ensures consistency and coverage across large data sets.
    - Integrates with notification or case-management systems for follow-up.

---

2. Architecture Overview

1. **Configuration**: Define SIEM credentials, IOC feeds, hunt queries, thresholds.
2. **Ingestion**: Pull IOCs or context from external sources (CSV, API).
3. **Hunt Execution**: Submit queries to SIEM via API, retrieve results.
4. **Analysis & Enrichment**: Filter, enrich results (e.g., geo-IP, asset tags).
5. **Reporting & Alerting**: Store hunt results, send notifications if findings exceed thresholds.
6. **Scheduling**: Run as a cron job, or integrate with workflow engine (e.g., Airflow).

---

3. Example Implementation Using Splunk SDK

3.1 Install Dependencies

```
pip install splunk-sdk requests
```

3.2 Configuration (YAML)

```yaml
# config.yaml
splunk:
  host: splunk.example.com
  port: 8089
  username: hunter
  password: hunter_password
hunt:
  ioc_feed_url: https://threat-feed.example.com/iocs.json
  searches:
    - name: "Rare PowerShell Invocations"
      query: 'search index=windows sourcetype=powershell | stats count BY user, CommandLine | where count < 5'
    - name: "Recent External DNS Queries"
      query: 'search index=dns sourcetype=dns_query | stats count BY query_name | where
query_name!="internal.company.local"'
  alert_threshold: 0   # number of results above which to alert
notification:
  slack_webhook: https://hooks.slack.com/services/XXX/YYY/ZZZ
```

3.3 Python Code

```python
import yaml
import requests
import splunklib.client as client
import splunklib.results as results
from datetime import datetime
```

```python
# 1. Load configuration
with open("config.yaml") as f:
    config = yaml.safe_load(f)

splunk_cfg = config["splunk"]
hunt_cfg   = config["hunt"]
noti_cfg   = config["notification"]

# 2. Authenticate to Splunk
service = client.Service(
    host=splunk_cfg["host"],
    port=splunk_cfg["port"],
    username=splunk_cfg["username"],
    password=splunk_cfg["password"]
)
if not service.is_authenticated():
    raise RuntimeError("Unable to authenticate to Splunk")

# 3. Fetch IOCs (if needed for enrichment)
def fetch_iocs(url):
    """
    Retrieve IOCs from external feed (e.g., JSON list of malicious IPs).
    """
    resp = requests.get(url, timeout=10)
    resp.raise_for_status()
    return resp.json()  # assume JSON array of strings

iocs = fetch_iocs(hunt_cfg["ioc_feed_url"])

# 4. Execute a hunt (search) and retrieve results
def run_hunt(name, query, earliest="-24h", latest="now"):
    """
    Submit a blocking Splunk search job and return list of result dicts.
```

```
    """
    # Prep a time-bounded search
    search_str = f"search {query} earliest_time={earliest} latest_time={latest}"
    job = service.jobs.create(search_str, exec_mode="blocking")
    reader = results.JSONResultsReader(job.results())
    records = [dict(item) for item in reader if isinstance(item, dict)]
    return records

# 5. Enrich and evaluate hunt results
def enrich_and_alert(hunt_name, records):
    """
    Enrich results (e.g., tag IOC hits) and send notification if above threshold.
    """
    count = len(records)
    if count <= hunt_cfg["alert_threshold"]:
        return  # nothing to alert

    # Optional enrichment: tag any IOC matches in fields
    for rec in records:
        for field, val in rec.items():
            if val in iocs:
                rec.setdefault("ioc_matched", []).append(val)

    # Build notification payload
    message = {
        "text": f"*Hunt:* {hunt_name}\n*Time:* {datetime.utcnow().isoformat()}Z\n*Results:* {count} records"
    }
    # Include first few records in message
    message["attachments"] = [{
        "title": "Sample records",
        "fields": [
            {"title": k, "value": str(v), "short": True}
            for k, v in list(records[0].items())[:5]
        ]
```

```
    }]

    # Send to Slack
    resp = requests.post(noti_cfg["slack_webhook"], json=message, timeout=5)
    resp.raise_for_status()

# 6. Orchestrator: iterate all hunts
def main():
    for hunt in hunt_cfg["searches"]:
        name  = hunt["name"]
        query = hunt["query"]
        print(f"Running hunt: {name}")
        results = run_hunt(name, query)
        print(f"  Found {len(results)} records")
        enrich_and_alert(name, results)

if __name__ == "__main__":
    main()
```

---

4. Key Teaching Points

- **Blocking vs. Non-Blocking**: `exec_mode="blocking"` simplifies small searches; large data sets require asynchronous polling.
- **Time Bounds**: Always specify `earliest_time` / `latest_time` to avoid re-scanning historical data.
- **IOC Enrichment**: Matching results against IOCs adds context without manual effort.
- **Thresholds**: Suppression thresholds prevent alert fatigue.
- **Notification Abstraction**: Encapsulate Slack/Email/Teams calls for easy replacement.
- **Configuration-Driven**: All hunts, thresholds, endpoints defined in YAML for maintainability.

- **Error Handling**: Raises on HTTP/Splunk errors; production code should implement retry/back-off.